

个性化JIT-SDP模型定制

个性化JIT-SDP模型定制

研究动机

- Developer Aware
- Cluster Aware
- Project Aware

核心方法

- 工作流程
- 基于JITFine修改的底模
- 模型微调
 - Base模型微调
 - Lora训练
- 聚类算法
 - 特征聚类和层次聚类
- Outlier Detector
 - Isolation Forest （孤立森林）
 - LOF （局部样本检测）
 - OneClass SVM
 - Mahalanobis （马氏距离）

权重叠加输出

- 网格法
- 梯度下降

Weigt简化求解

- 最小二乘法
- 逻辑回归
- 最优化求解
- 结果对比

RQ设置

- RQ1: 方法的有效性验证 (Does our Style-Aware DP outperform state-of-the-art defect prediction approaches?)
- RQ2: 验证各个模块的有效性 （Does every module in our model work?）
- RQ3:效率Effort-Aware （Is our method more cost-effective than the state-of-the-art defect prediction approaches?）
- RQ4: 适用场景 (In what scenarios is our approach suitable for?)

实验设置

- 数据集
- 底模训练
- Lora训练
 - Kmean
 - Project-aware
 - Developer-aware

性能比较

结果展示

- RQ1: StyleAware方法是否是有效的?
 - RQ1.1 在整体上表现的情况如何?
 - RQ1.2 模型分别在不同簇上的性能提升
- RQ2: StyleAware方法中各个模块是否都是Work的?
- RQ3: 工作量感知指标， Effort--Aware评估讨论

研究动机

Developer Aware

我们最开始想的是，利用大模型微调技术给每个**开发者**，定制适配自己代码风格的SDP模型，从而提高其预测的准确率。然而，这一个目标存在一些阻碍：

--2025/07/09新增小标题--

- **模型数量过多**：在一个大型的SDP项目数据集中，我们能发现数百个开发者，如果给每个开发者都订制模型会导致最终我们会产生很多模型。这会导致模型的选择适配过程变得相当复杂，反而无法给开发者带来便利，这显然违背了我们的初衷。
- **数据集极不均衡**：有些开发者占总数据的大部分，而有的开发者仅仅只出现过一次，我们只能在部分数据量充足的开发者之上构建微调模型，而事实上，“数据量充足”这个条件会筛掉90%的开发者。
- **数据集条件复杂**：训练集、测试集、评估集都是已经划分好了的，那么就显然会存在一种情况：训练集中所存在的一个开发者无法在另外的数据集中找到其对应（因为该数据集划分也不是按照我们的需求来的），这种情况需要进一步筛选，选择同时拥有训练集、测试集、评估集的开发者，那么最后满足条件的开发者就更少了。

基于上述种种原因，我们**修改了**仅仅依赖开发者的思路，转而进一步尝试别的思路。

Cluster Aware

第二种思路是聚类思路，简单来说就是根据其输入数据（**data level clustering**）进行聚类，然后对于每个聚类训练其特定的适配器。

这种方法也面对两个问题：

- **聚类特征选择**：SDP包含很多Feature，我们选择哪些数据作为聚类所使用的特征呢？
- **聚类方法选择**：我们该选择何种聚类方法？

这部分我们尝试了专家特征和语义特征，最后发现专家特征的表现更优于语义特征（聚类层面）。

聚类方法上，我们做了一些探索，首先是简单使用K-Mean进行聚类。结果显示，无论我们如何更改K-Mean聚类中K的值，最终的表现结果仍然堪忧，因此我们试着使用DBSCAN去寻找最佳的聚类方式，结果却显示专家特征的分布是异常密集的，因此无法适用于聚类策略。（同时轮廓系数的得分也很低）。

Project Aware

第三种思路是按照project对数据进行划分，为每个project训练其专有的适配器，这种划分方法很好地回避了Developer Aware所面对的问题：不存在数据集分布十分不均衡的现象，几乎每个Project都满足划分适配器的条件。事实上这种策略确实能很好地Work。

核心方法

工作流程

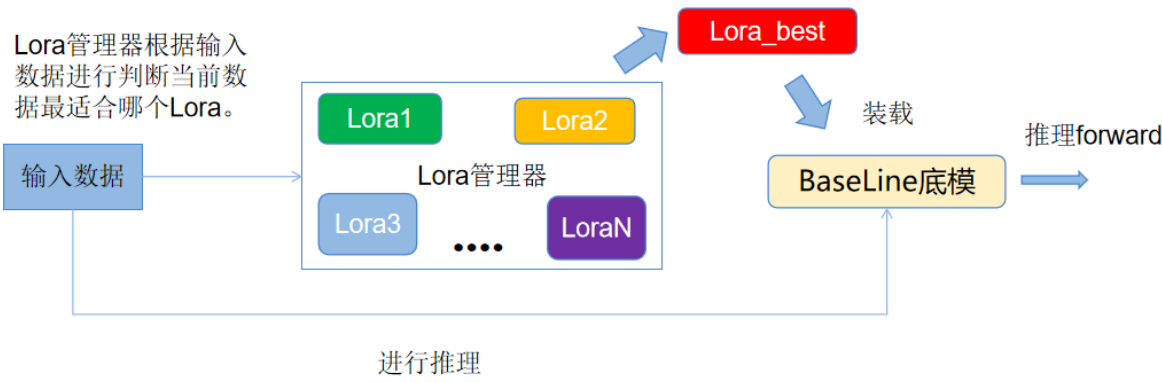
本项目构想的模型工作流程具体如下：

--2025/07/09新增列表小标题--

- 1. **底模训练**：构建能够在整体数据集上完成SDP任务的模型，作为底模。
- 2. **适配器训练**：分别训练特定的适配器（本实验用的Lora）。
- 3. **推理**：根据样本的属性选择合适的适配器载入底模进行预测，以达到提高准确率的目的。

推理测试部分

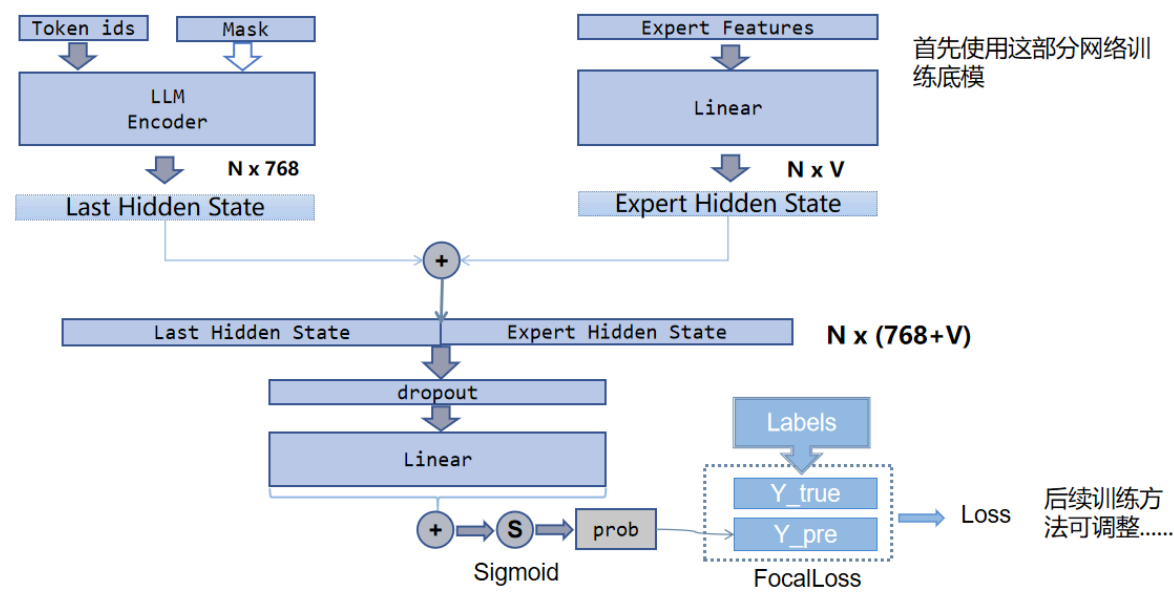
把底模和Lora管理分别交给不同的模块，而不是一个Class全部处理。



基于JITFine¹修改的底模

这份工作的逻辑是使用大模型技术对代码信息进行编码，然后通过下游全连接层实现预测任务（二分类预测，最后输出是Sigmoid层，设置0.5为正负阈值）。

底模是在JITFine¹模型的基础上进行了一些修改，其整体框架如图所示：



其中LLM Encoder部分指的是编码器部分，在JITFine中仅使用了基于Bert微调的CodeBert作为其编码器，编码器能够将代码和commit message的文本统一编码为中间向量（768维度），其中我们取开始符[CLS]的编码向量作为模型全连接层的输入。此外，对于Expert Feature。在对其进行归一化处理使用全连接层对其进行升维（14维feature到V维隐向量），然后与编码器的输出进行连接，最后将其输入到

预测层。

除了上述使用CodeBert作为编码器的情况，我们在另一份工作中发现他们额外使用了别的编码器，并且均能取得不错的表现²。

其额外的编码器包括**GraphCodeBert**，**PLBART**，**UniXcoder**，**CodeT5**。

此外，我们修改了其损失函数部分，将其原来的**BCELoss**换成了能更好处理难数据的**FocalLoss**。附上代码如下：

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class RobertaClassifier(nn.Module):
    def __init__(self, args):
        super().__init__()
        self.args = args
        self.manual_dense = nn.Linear(args.manual_feature_size, args.hidden_size)
        self.dropout = nn.Dropout(args.dropout)
        self.cat_proj = nn.Linear(2 * args.hidden_size, 1)

    def forward(self, features, manual_features):
        cls_features = features[:, 0, :]
        manual_features = manual_features.float()
        manual_features = self.manual_dense(manual_features)
        if self.args.activation == "tanh":
            manual_features = torch.tanh(manual_features)
        elif self.args.activation == "relu":
            manual_features = torch.relu(manual_features)

        cat_features = torch.cat((cls_features, manual_features), dim=1)
        cat_features = self.dropout(cat_features)
        proj_score = self.cat_proj(cat_features)
        return proj_score

class ConcatModel(nn.Module):
    def __init__(self, encoder, config, tokenizer, args):
        super(ConcatModel, self).__init__()
        self.encoder = encoder
        self.config = config
        self.tokenizer = tokenizer
        self.args = args
        self.classifier = RobertaClassifier(args)

    def forward(self, input_ids, input_mask, manual_features, label):
        if self.args.pretrained_model in ["codebert", "graphcodebert", "unixcoder"]:
            if self.args.base_train:
                outputs = self.encoder.base_model.model(input_ids=input_ids,
                attention_mask=input_mask)
            else:
                outputs = self.encoder(input_ids=input_ids,
                attention_mask=input_mask)
            elif self.args.pretrained_model in ["codet5"]:
```

```

        outputs = self.encoder.encoder(input_ids=input_ids,
attention_mask=input_mask)
        elif self.args.pretrained_model in ["plbart", "plbart-large"]:
            if self.args.base_train:
                outputs =
self.encoder.base_model.model.model.encoder(input_ids=input_ids,
attention_mask=input_mask)
            else:
                outputs = self.encoder.model.encoder(input_ids=input_ids,
attention_mask=input_mask)

        logits = self.classifier(outputs[0], manual_features)

        prob = torch.sigmoid(logits)

        if self.args.loss_fct == "bce":
            loss_fct = nn.BCELoss()
        elif self.args.loss_fct == "focal":
            loss_fct = FocalLoss()
        else:
            raise ValueError("Unsupported loss function!")

        loss = loss_fct(prob, torch.unsqueeze(label, dim=1).float())

        return prob, loss

```

```

class FocalLoss(nn.Module):
    def __init__(self, alpha=0.9, gamma=2, eps=1e-8):
        """
        Focal Loss 实现，适用于类别不平衡的二分类问题。

        参数:
            alpha (float): 类别1的权重，范围[0, 1]。默认0.75，适用于类别1为少数类的情况。
            gamma (float): 调节难易样本的因子，默认2。
            eps (float): 数值稳定性参数，防止log(0)。
        """
        super(FocalLoss, self).__init__()
        self.alpha = alpha
        self.gamma = gamma
        self.eps = eps

    def forward(self, prob, target):
        """
        参数:
            prob (Tensor): 预测的概率值（属于类别1的概率），形状为(batch_size, )
            target (Tensor): 真实标签，0或1，形状与prob相同。

        返回:
            Tensor: 计算后的Focal Loss。
        """
        # 确保输入形状一致
        prob = prob.view(-1)
        target = target.view(-1).float() # 转换为float以匹配where的条件

        # 数值稳定性处理，防止log(0)

```

```

prob = torch.clamp(prob, self.eps, 1 - self.eps)

# 计算p_t: 当target=1时取prob, 否则取1-prob
p_t = torch.where(target == 1, prob, 1 - prob)

# 计算交叉熵损失项
ce_loss = -torch.log(p_t)

# 计算alpha因子: 类别1使用alpha, 类别0使用1-alpha
alpha_t = torch.where(target == 1, self.alpha, 1 - self.alpha)

# 计算调制因子和总损失
focal_loss = alpha_t * torch.pow(1 - p_t, self.gamma) * ce_loss

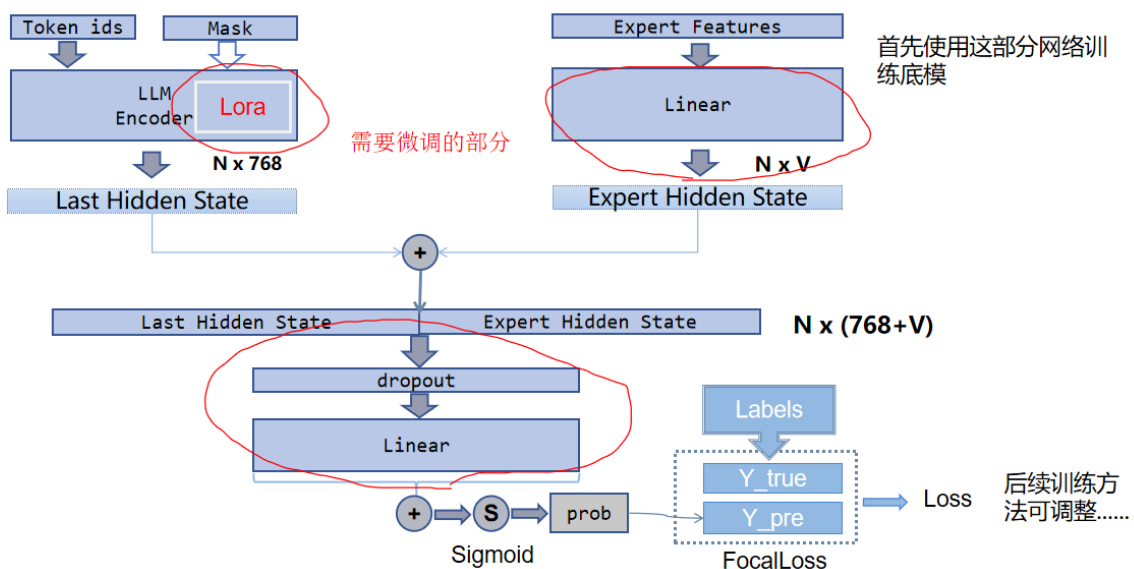
# 返回平均损失
return focal_loss.mean()

```

模型微调

Base模型微调

首要目标是训练底模以达到在整体数据集上能够预测，根据相关工作表明²，我们可以仅对模型的部分进行微调以达到训练底模的目的，如图所示，我们不需要训练整个编码器，这个开销过于巨大且耗时非常久。在训练底模阶段，我们只需要对编码器的注意力层添加Lora，然后训练该Lora和其他部分的全连接层即可。



具体的，我们需要使用到PEFT这个Python包进行实现，Peft包含了使用Python训练Lora所必要的接口，具体代码可以展示如下：

```

from peft import LoraConfig, TaskType, get_peft_model
#####假设model是获取到的Encoder#####
#####对于不同的Encoder, 选择嵌入注意力层的名称不同#####
if args.pretrained_model in ["codet5"]:
    peft_config = LoraConfig(inference_mode=False, r=64, lora_alpha=32,
                             lora_dropout=0.1, target_modules=["q", "v"])
elif args.pretrained_model in ["plbart"]:

```

```

peft_config = LoraConfig(inference_mode=False, r=64, lora_alpha=32,
                        lora_dropout=0.1, target_modules=["q_proj",
"v_proj"])
elif args.pretrained_model in ["codebert", "graphcodebert", "unixcoder"]:
    peft_config = LoraConfig(inference_mode=False, r=64, lora_alpha=32,
                        lora_dropout=0.1, target_modules=["query", "value"])
model = get_peft_model(model, peft_config)

mymodel = MyModel(model, config, tokenizer, args).to(device)
#####后续直接将mymodel送入torch的优化器即可，PeftModel只会更新目标层，其他层的梯度会
被冻结

```

Lora训练

对于底模之上的Lora训练，训练参数可以做到更少，只需要对编码器的注意力层和Expert Feature转换层和输出层这三个部分嵌入Lora即可。代码进一步修改如下：

```

mymodel = ConcatModel(model, config, tokenizer, args).to(device)
target_modules = []
if args.pretrained_model in ["codet5"]:
    target_modules += ["q", "v"]
elif args.pretrained_model in ["plbart"]:
    target_modules += ["q_proj", "v_proj"]
elif args.pretrained_model in ["codebert", "graphcodebert", "unixcoder"]:
    target_modules += ["query", "value"]
if args.lora_train:
    if args.base_model == "single":
        target_modules += ["ll_proj"]
    elif args.base_model == "concat":
        target_modules += ["manual_dense", "cat_proj"]
    elif args.base_model == "manual":
        target_modules += ["manual_dense", "ll_proj"]

peft_config = LoraConfig(inference_mode=False, r=64, lora_alpha=32,
                        lora_dropout=0.1, target_modules=target_modules)
mymodel = get_peft_model(mymodel, peft_config)

```

聚类算法

特征聚类和层次聚类

最终，我们的策略是在首次聚类的基础上进一步对聚好的集合再次进行聚类，其出发点如下：

- 首先，我们发现无论是developer还是project策略的聚类最终都会得到很多cluster，这导致数据被过于分散，这种情况下会导致训练Lora的数据不足，从而影响模型最终的表现性能。我们的想法是能否有一种方案使数据更加密集，因此选择了对集合再次进行聚类。
- 其次，这种方法能解决developer aware策略中对应developer缺少数据的问题。

具体策略如下：

对于project aware，我们首先按照project的名字对其进行划分，得到一组集合，对于这些集合，我们对其进一步使用Wasserstein距离计算所有集合在这十四个特征的距离，取14个特征的平均距离作为集合的距离，最终能得到距离矩阵（任意一集合到另一个集合的距离）。

```

from scipy.stats import wasserstein_distance
from scipy.spatial.distance import squareform

def pairwise_wasserstein(collections):
    """Wasserstein distance between datasets"""
    n = len(collections)
    dist_matrix = np.zeros((n, n))

    for i in range(n):
        for j in range(i+1, n):
            # Use the mean wasserstein distance among the 14 features.
            dists = [wasserstein_distance(collections[i][:,d], collections[j]
[:,d])
                    for d in range(collections[i].shape[1])]
            dist_matrix[i,j] = np.mean(dists)
            dist_matrix[j,i] = dist_matrix[i,j]
    return squareform(dist_matrix)

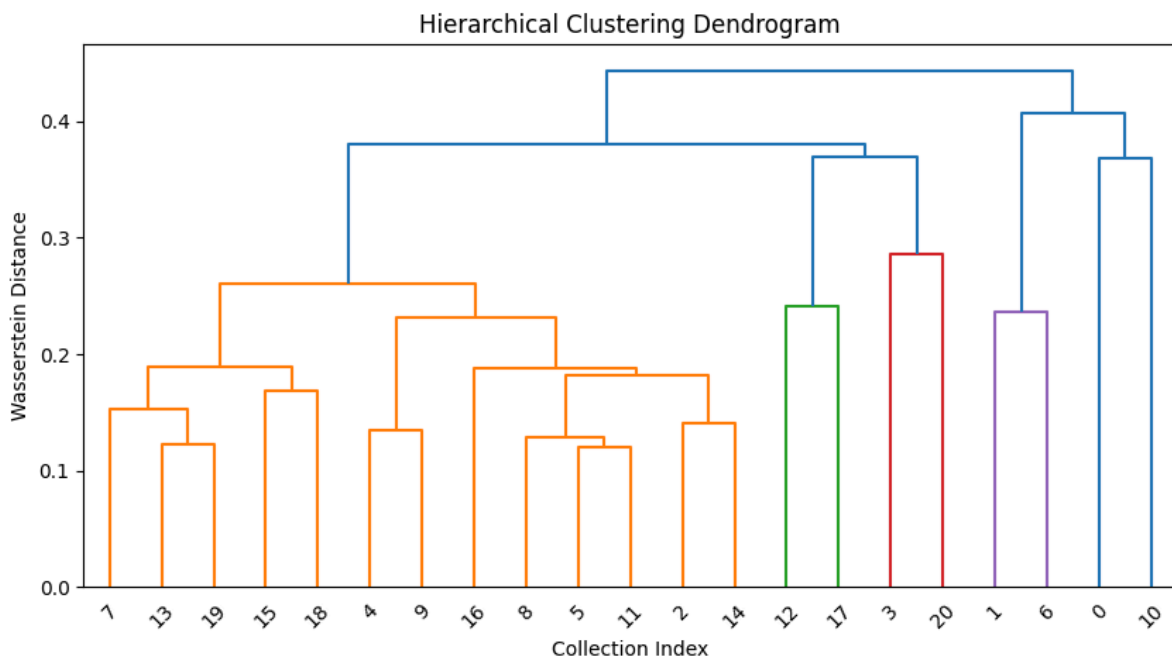
```

在这之后，使用层次聚类进行集合直接的再次聚类，由于已经获得距离矩阵，可以很容易地得到层次聚类的结果。

```

from scipy.cluster.hierarchy import linkage, fcluster, dendrogram
distances=pairwise_wasserstein(collections)
Z=linkage(distances,method="average")
clusters=fcluster(Z,t=self.n_cluster,criterion="maxclust")
#####clusters是一个列表，其内容为collections的label，呈现一一对应关系。
#####Display
dendrogram(Z)

```



Outlier Detector

我们发现，加入Lora后，**有些数据集的表现性能反而下降了**，但是理论上来说不该这样：因为我们在训练Lora的时候用最小Loss来做的早停策略，只会保存训练时性能最好的模型（即最坏情况也是和底模性能一样），所以不应该会导致性能反而下降。

唯一的可能性是，在训练阶段使用早停策略时，valid数据集和test的数据集分布差异较大，所以在valid上表现更好的样本在test上会出现性能反而下降的情况。

为了解决这一问题，一个很简答的思路是，我们通过某种策略找出test数据集中明显分布与valid集不同的点，即outlier（离群点）。也叫离群点检测。下列给出简答的离群点检测思路（也基于14个feature进行检验）：

Isolation Forest（孤立森林）

孤立森林是工业界中常用的一种离群点检测机制，其核心思想是认为离群点比正常点更加容易被分离。在使用孤立树时，每次孤立树会随机选择一个特征进行划分，将数据划分为两类，重复该过程知道所有叶子节点上均只包含一个样本（每个样本被划分为一个元数据）。孤立森林认为，最早被挑选出来的数据（深度更小）是离群点的概率更大，并且基于其深度。而孤立森林就是base learner是孤立树的森林。

在实践中，显然不可能对所有的样本都划分到元数据，因此，孤立森林往往会随机抽样，每个孤立树只会收到部分样本，且不会一直划分所有样本，通常得到预期占比的样本后就会停止划分。调用代码如下：

```
from sklearn.ensemble import IsolationForest

clf = IsolationForest(
    contamination=0.1, #超参，实验表明0.1比价好
    random_state=self.args.seed
)

clf.fit(samples)

outlier_mask=clf.predict(test_sample) == -1 # -1表示异常

# outlier_mask是形如[True,False,True,False.....]的遮罩(np.ndarray)。
```

LOF（局部样本检测）

```
from sklearn.neighbors import LocalOutlierFactor

lof = LocalOutlierFactor(
    contamination="auto",
    novelty=True
)

lof.fit(samples)

outlier_mask=lof.predict(test_sample) == -1 # -1表示异常
```

OneClass SVM

```
from sklearn.svm import OneClassSVM

svm=OneClassSVM(nu=0.1, kernel="rbf", gamma="auto")
svm.fit(train)

outlier_mask= svm.predict(test_sample)==-1
```

Mahalanobis (马氏距离)

```
from sklearn.covariance import MinCovDet

robust_cov =
MinCovDet(random_state=self.args.seed, support_fraction=0.8).fit(train)

mahal_dist = robust_cov.mahalanobis(test_sample)
# 计算点到鲁棒中心的距离，超过阈值就认为是离群点
# 计算阈值 - 使用卡方分布
n_features = 14
threshold = chi2.ppf(0.9999, df=n_features)

# 标记离群点
outliers = mahal_dist > threshold
```

权重叠加输出

加入离群点检测后，我们发现，“加入Lora后反而性能下降”这个问题得到了一定的缓和，但是随之而来的是新的问题，我们发现**原本某些Lora性能非常好的结果出现了一定的下降**，这是因为离群点检测把很多正常点（在Lora上表现更好的点）错误判断成了离群点，导致过多的点被错误交给了底模进行预测，这一问题可以通过修改离群点检测的超参和策略来得到缓和，此处提出一种Soft方法也来处理这个问题。我们仿照集成模型的思路，对于outlier点，不是直接扔给底模，而是取底模和Lora两组模型的加权求和（因为输出是一个概率），这样对于误报的outlier，只需要将权重设置很大，让原Lora的占比更多；对于真的outlier，就让Lora的预测占比更小即可。新的预测结果可以表示为：

$$prob = (1 - weight) * BaseModel(x) + weight * LoraModel(x)$$

于是找到一个合理的weight成了我们需要解决的问题。

网格法

设置weight = [0,1]，不断试探weight为不同值时的模型性能，从而找到最好的值。

这种方法存在明显的问题，首先划分weight的间隔是否合理？让模型重复运行大量的实验会浪费很多资源，且什么叫最好的值？同一组weight在不同指标（f1, recall, mcc, gmean）上的表现可能各不相同，选择什么作为基准也是一个问题。

梯度下降

设置weight初始值0.5，用valid数据集作为此处的训练集，不断梯度下降以找到最优解（损失函数设置为原本Focall Loss的叠加）

$$L_{weight} = (1 - weight) * L_{base} + weight * L_{lora}$$

Weight简化求解

-----2025/07/02更新-----

在上一节中，讨论了使用梯度下降求解weight的策略，这是一个很简单常用的策略，但是存在一个很明显的问题：开销过大。在我们的方法中，已经包含了底模训练和Lora训练两个大的训练步骤，现在又在prediction阶段为了获取一个新的参数再进行一轮训练，一共三次train显得非常冗余了。所以我们想尽可能简化这个部分，由于只包含w一个未知数和（P1,P2）两个输入，因此可以考虑用简单的方法代替梯度下降（其中P1代指底模的输出，P2代指Lora模型的输出）。

此处考虑到了线性回归**最小二乘法**求解，**逻辑回归拟合**以及**最优化**求解。最终选择使用最优化方法求解，下面给出具体的探索过程和结果：

最小二乘法

最小二乘法是求解这种线性拟合问题的一种经典方法，在我们的问题中，输出概率 $\text{Prob} = (1 - \text{weight}) * P_1 + \text{weight} * P_2$ ，其中 P_1, P_2 可以看作回归问题的输入，二分类问题的标签 label 是一个 $\{0, 1\}$ 二值，我们可以将其看成回归问题，即1类的 Prob 尽可能接近1.0，0类的 Prob 尽可能接近0.0。这样就能对其构建损失函数（均方误差）：

$$\text{Loss} = \frac{1}{n} \sum_{i=0}^n ((1 - \text{weight}) * P_1^i + \text{weight} * P_2^i - y^i)^2 \quad (1)$$

对公式（1）展开后化简得到（2）

$$\text{Loss} = \frac{1}{n} \sum_{i=0}^n (\text{weight} * (P_2^i - P_1^i) + P_1^i - y^i)^2 \quad (2)$$

令

$$A = P_2^i - P_1^i \quad (3.1)$$

$$B = P_1^i - y^i \quad (3.2)$$

带入到(2)中得到

$$\text{Loss} = \frac{1}{n} \sum_{i=0}^n (\text{weight} * A + B)^2 \quad (4)$$

对（4）关于 weight 求偏导得到

$$\frac{\partial \text{Loss}}{\partial \text{weight}} = \frac{2}{n} \sum_{i=0}^n (\text{weight} * A + B) A = 0$$

化简后得到

$$\text{weight} * \sum_{i=0}^n A^2 = \sum_{i=0}^n AB$$

$$\text{weight} = \frac{\sum_{i=0}^n AB}{\sum_{i=0}^n A^2}$$

$$\text{weight} = \frac{\sum_{i=0}^n (P_2^i - P_1^i)(P_1^i - y^i)}{\sum_{i=0}^n (P_2^i - P_1^i)^2}$$

根据公式所示，可以求得 weight 的值，但是，实验显示，最小二乘法求解是不能解决我们此处的问题的。原因是，我们此处默认的 w 是属于 $[0, 1]$ 的一个小数，而此处求解 w 不存在约束，且是满足整个 valid 数据集的，所以可能会出现很奇怪的数值。此外，此处简单将分类问题看成回归问题，会导致一些很明显的错误，比如在实际处理中， $\text{Prob} > 0.5$ 就认为可以是1类了，但是此处却仍然会认为结果是错误的，与1.0有均方误差。实验结果也的确如此，得到的 weight 全是正负几千的异常值。显然不能用。

逻辑回归

在上节使用最小二乘法求解最优 weight 时，将分类问题看作了回归问题，离散类标看成了浮点数，损失了很多有效信息。有没有一种方法既能简单使用回归问题求解最优 weight ？又能考虑分类问题呢？此时用到了回归模型中的经典分类模型——Logistic Regression。

逻辑回归求解上述问题可看作

$$\text{Sigmoid}(A * P_1 + B * P_2)$$

其中逻辑回归会取阈值0.5作为判别标准，使用二元交叉熵损失作为其损失函数。最后会得到一组适用的A和B，经过恒等变换后可以得到目标weight，代码实现如下：

```
from sklearn.linear_model import LogisticRegression

X=np.concatenate([P1, P2], axis=1)
model = LogisticRegression(fit_intercept=False, penalty='l2')
model.fit(X, true_label)
# 计算分子和分母
# 获取权重系数
weights = model.coef_[0]
w = weights[1] / (weights[0] + weights[1]) # 计算等效的 w

print(f"逻辑回归权重: w1={weights[0]:.4f}, w2={weights[1]:.4f}")
print(f"等效权重 w = {w:.4f}")
```

关于逻辑回归也存在一定问题，虽然我们恒等变换之后能够将w转换到小数级别，但是却未约束w的正负，在实验中会出现weight为负数的情况，显得很奇怪。

最优化求解

最后一种方法是使用最优化进行求解，这种方法是近似求解的一种，但是速度比梯度下降快很多，且可以对weight的值进行约束，且使用FocalLoss作为损失函数（与我们训练一致）。给出代码如下：

```
from scipy.optimize import minimize

# 定义目标函数（均方误差）最优化方法
def objective(w):

    prob = (1 - w) * P1 + w * P2

    # 确保输入形状一致
    prob = prob.reshape(-1,1)
    target = true_label.astype(float).reshape(-1,1) # 转换为float以匹配where的条件

    # 数值稳定性处理，防止log(0)
    prob = np.clip(prob, 1e-8, 1 - 1e-8)

    # 计算p_t: 当target=1时取prob, 否则取1-prob
    p_t = np.where(target == 1, prob, 1 - prob)

    # 计算交叉熵损失项
    ce_loss = -np.log(p_t)

    # 计算alpha因子: 类别1使用alpha, 类别0使用1-alpha
    alpha_t = np.where(target == 1, 0.75, 1 - 0.75)

    # 计算调制因子和总损失
    focal_loss = alpha_t * np.pow(1 - p_t, 2) * ce_loss

    return np.sum(focal_loss)
```

```

# 设置约束  $w \in [0,1]$ 
bounds = [(0, 1)] # w必须在[0,1]范围内

# 初始猜测
initial_guess = 0.5

# 求解优化问题
result = minimize(objective, initial_guess,
                  method='L-BFGS-B', bounds=bounds)

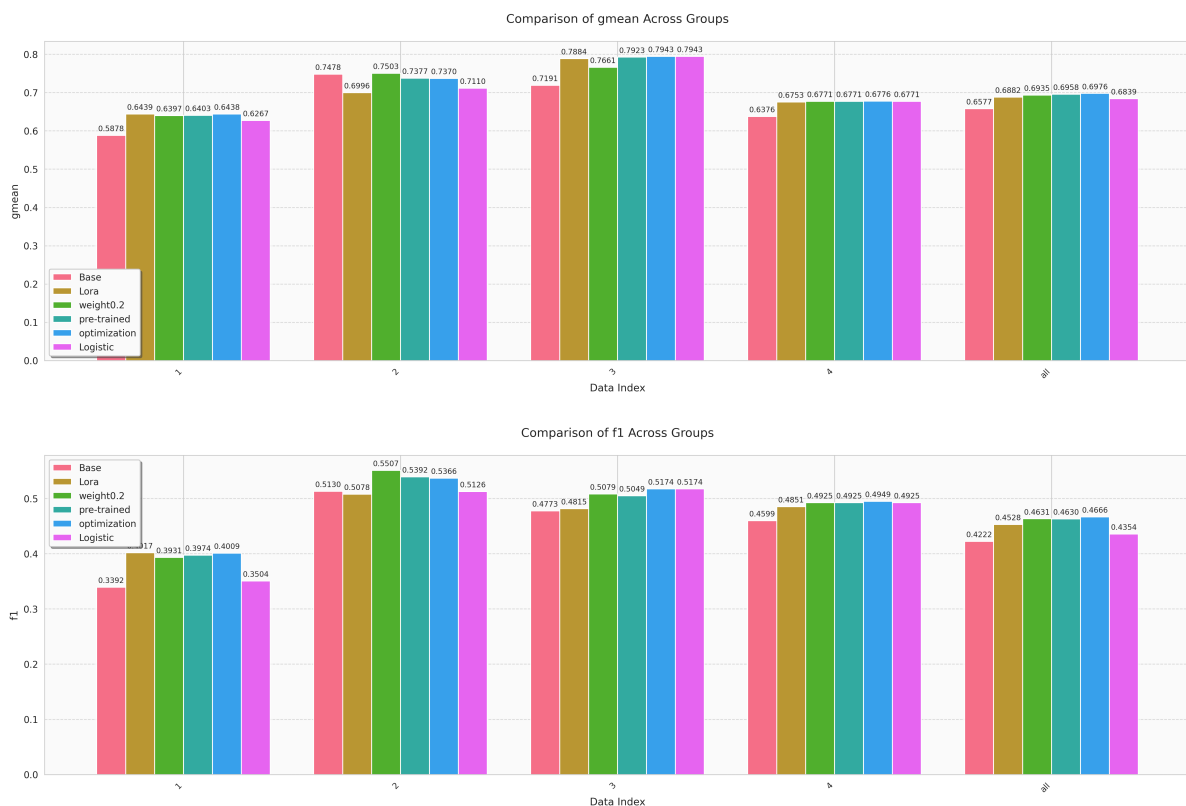
if result.success:
    w_optimal = result.x[0]
    print(f"最优权重 w = {w_optimal:.4f}")

    # 计算预测值
    # predictions = (1 - w_optimal) * P1 + w_optimal * P2
    # print("预测值:", predictions)
    weight_dict[project_name] = w_optimal
else:
    print("优化失败:", result.message)

```

结果对比

分别在base, 纯Lora, weight=0.2, weight=预训练 (pre-trained), weight=逻辑回归求解, weight=最优化 (optimization) 进行了比较实验, 结果如下:



显然最优化算法和梯度下降是最优的

RQ设置

——2025/07/20——

目前, 根据研究内容可以设置RQ如下:

RQ1: 方法的有效性验证 (Does our Style-Aware DP outperform state-of-the-art defect prediction approaches?)

RQ1是很自然的思路：首先是要验证方法的有效性。目前，我们初步考虑将三种Lora实现的方法都纳入到对比策略中：Developer，Project，Data。与其对比的底模是JITFine的微调模型（参考实验的底模，在参考实验中他们已经和JITLine和CC2VEC进行了对比实验，所以此处这两组对比可以当成备选案），于是对比实验的结果大致可以有这样的表格（指标可以选择F1和GMean）：

	*CodeBert***	*Plart***	*CodeGraph***	*UnixCoder***	*CodeT5***
JITFine-bce					
*JITFine-focal***					
*KMeans-Lora***					
*Developer-Lora***					
*Project-Lora***					
JITLine ³					
CC2VEC ⁴					
CCT5 ⁵					

其中包含底模和三种策略的表现性能（此处取整体表现作为目标数据）。额外的模型包括bce损失的底模，JITLine和CC2VEC，也可以加入到实验中。（此处用到的数据集是Defects4j，在参考实验中已经准备好了Defects4j关于JITLine和CC2VEC的版本和接口，可以直接使用，***我新找到的两个数据集暂时无法在这两种方法上使用，所以数据集的选取需要进一步考虑***）。

之前讨论Lora的策略是想让其在特定的数据集上表现更好，所以很自然包括关于划分Lora后其性能的变化评估。

这部分将分别对聚类后的每个簇进行单独分析，看加入Lora后整体性能在某个簇上的提升。这部分可以不涉及JITLine和CC2VEC，因此可以使用我们的数据集（Qt和OpenStack）。此处可以修改超参n（簇的个数）和其他超参（聚类算法的超参）进行多次实验，实验结果可以就用之前组会类似的图片展示：

RQ2: 验证各个模块的有效性 (Does every module in our model work?)

目前整个项目的模块可以归类为：

- **BaseModel**：底模（有三种：纯14个特征，纯语义特征和混合特征；以及五种编码器。实际上纯14个特征的底模基本上没有预测能力）。
- **ClusterManager**：聚类管理器。包含拟合聚类模型，对数据进行聚类划分以及挑选该样本对应的Lora等。
- **OutlierDetector**：离群点检测器。在prediction阶段，离群点检测器会判断当前的样本是否能用当前的Lora进行预测，如果判断其为离群点，我们认为此时用底模进行预测反而性能会更好。

- **WeightEnsemble**: 加权预测。在预测阶段，如果加入Lora。将对底模和Lora输出的结果进行加权求和，取最终的输出作为预测概率（权重通过在valid数据集上利用拟牛顿法进行最优化求解）。

核心模块是BaseModel和ClusterManager，这两部分的讨论已经由RQ1得到。我们在RQ2想通过消融实验说明另外两个模块的重要性以及作用。

RQ3:效率Effort-Aware (Is our method more cost-effective than the state-of-the-art defect prediction approaches?)

——2025/07/21——

软工相关的离线预测通常也会考虑到预测效率。一般的，在实际的软件开发过程中，用于修改缺陷的资源是有限的，我们不可能对整个数据集进行检验，通常只想对最有可能的几个样本提交进行检验，对应的指标有：

- **Precision@20%**是指检查 20%代码中找到的有缺陷变更占这 20%所包含全部代码变更的比例。
- **Recall@20%**是指检查 20%代码中找到的有缺陷变更占整个数据集中所有有缺陷变更的比例。
- **F1@20%**与第 3.2.1 节中 *F1-measure* 的定义类似,是 *Precision@20%*和 *Recall@20%*的调和平均,即

$$F1@20\% = \frac{2 * Precision@20\% * Recall@20\%}{Precision@20\% + Recall@20\%}$$

这部分需要额外讨论，因此不基于RQ1。

RQ4: 适用场景 (In what scenarios is our approach suitable for?)

此部分的目标是探索三种策略的适用场景。在数据集分布层面，比较三种策略的具体表现情况，从而给出三种策略的选择思路。

实验设置

数据集

目前暂定使用JITFine的数据集——Defects4J。其数据集具体内容如下图所示：

上述部分是有关Project的内容，在我们的方法中还包含Developer的内容如下：

我们取训练集中至少包含100组样本的开发者作为集合样本，其他开发者数据过于零散，只能忽略不计（如果将零散的开发者训练到一个Lora中显然是不合理的），在测试集中，我们同样认为开发者数量大于100的取集合，但和训练集不同，此时零散的开发者对应的样本将会交给底模处理。

训练集包含**193**个开发者

验证集包含**144**个开发者

测试集包含**344**个开发者

可以发现开发者的数量非常零散，有些开发者甚至只有一两条样本，很难构成比较好的数据集（大模型对数据集质量要求很高），因此这种层次聚类策略可以将比较相似的开发者聚在一起，从而提升样本数量。

Name	Train	Valid	Tes
Sebastian Bazley	1436	982	523
Stephen Colebourne	1161	\	1
Henri Yandell	870	101	3
Luc Maisonobe	847	167	288
Oliver Heger	833	422	349
Gary D. Gregory	774	605	473
Xavier Hanin	738	52	5
Phil Steitz	632	85	44
Stefan Bodewig	542	212	235
Gilles Sadowski	390	250	44
.....			

底模训练

此处仅选择同时考虑语义特征和专家特征的JITFine完整版模型，设置训练参数如下：

- **learning_rate=2e-5**
- **patiance=4**
- **epochs=10**

大模型训练默认最终结果都会收敛，所以底模训练不涉及多次实验（ML的随机性更大，且更快，往往会多次训练多个版本）。编码器选择为CodeBert，GraphCodeBERT，Unixcoder，Plbart，CCT5（参考实验中用的CodeT5，但是CCT5说自己也是基于T5模型的，且也包含编码器部分，所以目前考虑用CCT5代替CodeT5）。

额外的，对于 CCT5设置其FocalLoss的alpha为0.9，CodeT5对应的alpha为0.8。其余仍设置为0.75

Lora训练

对于三种方法，分别设置n_cluster=[4,6,8,10]来进行探索。设置

- learning_rate=1e-4
- patiance=2
- epochs=10

Kmean

对于Kmean策略，使用Kmean++进行初始化，使用训练集进行初始化，可以得到n个簇划分的数据集。每个数据集作为对应Lora的训练数据进行训练，能够得到n个Lora。在valid或test过程中，只需要用Kmean预测对应的簇即可，选择其目标作为对应的valid/test模型。

Project-aware

对于JITFine数据集，其包含22个Project，都有对应的train,valid和test数据。

按照Project进行划分可以得到22个小数据集，计算每个小数据集的Wasserstein距离，根据该距离进一步进行层次聚类，最终得到n个簇。

Developer-aware

选取样本量大于100的Developer作为层次聚类的小类，进一步得到n个大类。

性能比较

测试阶段，进行**五次**实验，取平均值作为最终的结果。

采用Gmean,F1和AUC指标作为评估指标（AUC是因为输出仍然是sigmoid，阈值是一个超参）。

同时对比工作量感知指标**Recall@20%， Precision@20%， F1@20%**。

Lora性能比较取性能最好的n（预实验是n=4）

消融实验主要测试outlier detector和weight策略。

结果展示

RQ1: StyleAware方法是否是有效的？

RQ1.1 在整体上表现的情况如何？

此处，将通过六组Encoder在引入StyleAware前后的性能差异以及和另外两个Deep模型性能对比，来验证方法的有效性。

首先，验证其在F1上的表现：

	CodeBert	Plbart	GraphCodeBert	UnixCoder	CodeT5	CCT5
JITFine-base	0.427	0.470	0.453	0.456	0.463	0.442
Kmean-Style	0.471	0.494	0.489	0.479	0.485	0.453
Developer-Style	0.443	0.479	0.464	0.466	0.468	0.452
Project-Style	0.465	0.493	0.470	0.464	0.475	0.463
CC2VEC	0.159	0.159	0.159	0.159	0.159	0.159
JITLine	0.261	0.261	0.261	0.261	0.261	0.261

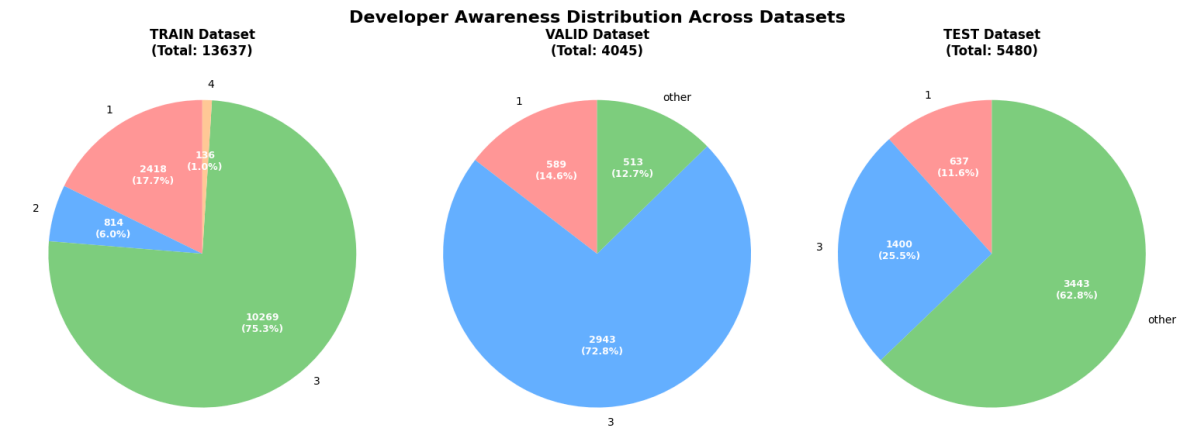
该结果表明，在F1指标上，Kmean的表现是最好的，其次是Project方法。可以发现Developer方法的表现性能较差，这可能是由于数据量不足导致的，一方面，Developer Aware种仅有少量有效样本可以用作训练Lora，另一方面，在测试集中也只有少量的测试样本能够找到其对应的Lora，这导致了实际测试集中能够被Developer Aware所处理的数据很少。

其次，在Gmean上的表现如下：

*	CodeBert	Plart	GraphCodeBert	UnixCoder	CodeT5	CCT5
JITFine-base	0.659	0.671	0.696	0.682	0.695	0.687
KMeans-Lora	0.733	0.744	0.746	0.734	0.748	0.741
Developer-Lora	0.670	0.685	0.706	0.698	0.693	0.702
Project-Lora	0.734	0.750	0.749	0.707	0.750	0.743
JITLine	0.401	0.401	0.401	0.401	0.401	0.401
CC2VEC	0.	0	0.	0	0.	0

在Gmean中，Project方法的表现更好，但是在Developer上的性能也比较差。

此处给出Developer划分用到的数据集占比（有效数据和无效数据）



不难发现，在实际训练中，Lora2和4占比都非常小，属于比较无效的数据，所以在评估和测试中只能找到1，3对应的数据。观察第三个表格可以发现，最后测试只用了约36%的样本，剩下62%的样本都被归类到others里面了，这导致了实际性能提升不太多，那么如果只考虑1，3簇单独的性能提升会如何呢？

结果显示： **Todo**

RQ1.2 模型分别在不同簇上的性能提升

我们方法的核心是对于不同的代码风格都能为其训练一个有效的适配器Lora，因此除了关注总的性能，我们更关心在每个簇上的表现。此处，将列出六个模型分别在不同簇上的性能变化。

第一个表格是**Kmean(F1)**

	Cluster 1	Cluster 2	Cluster 4	Cluster 3
codebert	0.27->0.33	0.57->0.57	0.38->0.47	0.5->0.54
plbart	0.28->0.4	0.61->0.6	0.41->0.46	0.57->0.56
graphcodebert	0.3->0.36	0.61->0.62	0.44->0.48	0.53->0.56
unixcoder	0.35->0.37	0.59->0.63	0.37->0.43	0.56->0.6
codet5	0.32->0.41	0.62->0.61	0.43->0.43	0.54->0.55

	Cluster 1	Cluster 2	Cluster 4	Cluster 3
cct5	0.32->0.31	0.68->0.68	0.42->0.46	0.55->0.57

第二个表是 Kmean (Gmean)

	Cluster 1	Cluster 2	Cluster 4	Cluster 3
codebert	0.46->0.55	0.73->0.73	0.55->0.71	0.8->0.83
plbart	0.44->0.64	0.76->0.76	0.56->0.65	0.83->0.84
graphcodebert	0.53->0.6	0.77->0.78	0.63->0.7	0.81->0.84
unixcoder	0.57->0.65	0.75->0.79	0.56->0.64	0.81->0.83
codet5	0.54->0.65	0.77->0.77	0.62->0.68	0.81->0.83
cct5	0.62->0.66	0.81->0.81	0.6->0.68	0.78->0.83

Project (F1)

	Cluster 1	Cluster 2	Cluster 4	Cluster 3
codebert	0.34->0.42	0.54->0.53	0.48->0.48	0.47->0.5
plbart	0.4->0.42	0.59->0.59	0.51->0.52	0.48->0.55
graphcodebert	0.36->0.39	0.55->0.55	0.52->0.54	0.51->0.54
unixcoder	0.4->0.4	0.56->0.57	0.48->0.53	0.46->0.46
codet5	0.36->0.41	0.56->0.56	0.52->0.56	0.55->0.5
cct5	0.38->0.39	0.45->0.47	0.5->0.59	0.5->0.54

Project (Gmean)

	Cluster 1	Cluster 2	Cluster 4	Cluster 3
codebert	0.58->0.7	0.76->0.77	0.71->0.79	0.65->0.71
plbart	0.62->0.72	0.78->0.79	0.67->0.79	0.65->0.74
graphcodebert	0.64->0.72	0.78->0.78	0.74->0.79	0.69->0.75
unixcoder	0.64->0.67	0.78->0.77	0.68->0.77	0.66->0.66
codet5	0.63->0.72	0.77->0.78	0.75->0.84	0.71->0.7
cct5	0.65->0.71	0.72->0.76	0.71->0.82	0.67->0.72

Developer (F1)

	Cluster 1	Cluster 3
codebert	0.5->0.58	0.28->0.34
plbart	0.47->0.52	0.41->0.44
graphcodebert	0.51->0.55	0.35->0.38
unixcoder	0.38->0.45	0.34->0.38
codet5	0.47->0.5	0.36->0.37
cct5	0.55->0.6	0.36->0.39

Developer (Gmean)

	Cluster 1	Cluster 3
codebert	0.65->0.73	0.5->0.54
plbart	0.58->0.65	0.6->0.64
graphcodebert	0.68->0.75	0.58->0.6
unixcoder	0.54->0.63	0.56->0.61
codet5	0.63->0.67	0.59->0.55
cct5	0.7->0.75	0.56->0.61

整体来看，在所有簇上的表现也均不错。

对于Kmean，一共有6个模型*4=24个簇，在F1上仅三个表现更差，Gmean上则全部表现更好。

在Project上，F1和Gmean都只有两组表现更差。

Developer上则全部表现更好。

综上所述，StyleAware在簇上是有效的。

RQ2: StyleAware方法中各个模块是否都是Work的？

回顾本方法的框架如下：



我们的方法可以分为两个模块：

- Lora划分和适配模块：此模块将训练数据划分到不同簇上进行训练，并为测试数据找到最适合其的Lora。
- OutlierDetector模块：此模块检测测试数据中的异常点，对于这些异常点，将降低其依赖Lora的权重，进行一个Smooth的预测。

对于Lora划分和适配模块，可以对比其Base和加入Lora后的性能差异（F1），

	codebert	plbart	graphcodebert	unixcoder	codet5	cct5
developer_aware	0.43->0.45	0.47->0.48	0.45->0.47	0.46->0.47	0.46->0.47	0.44->0.45
Kmean	0.43->0.47	0.47->0.48	0.45->0.49	0.46->0.48	0.46->0.48	0.44->0.45
project_final	0.43->0.46	0.47->0.49	0.45->0.47	0.46->0.47	0.46->0.47	0.44->0.47

结果显示，Lora思路确实是合理的，并且能有效地提升模型的性能。

进一步地，对比在Lora模块上加入OutlierDetector后的性能变化。

	codebert	plbart	graphcodebert	unixcoder	codet5	cct5
developer_aware	0.45->0.44	0.48->0.48	0.47->0.46	0.47->0.47	0.47->0.47	0.45->0.45
Kmean	0.47->0.47	0.48->0.49	0.49->0.49	0.48->0.48	0.48->0.49	0.45->0.45
project_final	0.46->0.47	0.49->0.49	0.47->0.47	0.47->0.46	0.47->0.48	0.47->0.46

结果显示，OutlierDetector能在一定程度上提升模型性能的鲁棒性，但是会导致某些模型的表现性能下降。

最初考虑使用OutlierDetector是因为某些簇上加入Lora后性能反而下降，想去缓和这个下降的趋势。

下面给出在每个簇上的性能差异 (F1)：

Kmean	0	1	2	3
codebert	0.2659->0.3308->0.3279	0.5724->0.5727->0.5727	0.3848->0.4754->0.4729	0.504->0.5377->0.5375
plbart	0.2802->0.3779->0.4012	0.6085->0.5999->0.5999	0.4094->0.4618->0.4618	0.5663->0.5477->0.5624
graphcodebert	0.3015->0.3579->0.3634	0.6125->0.622->0.6221	0.4397->0.477->0.477	0.5267->0.558->0.5568

Kmean	0	1	2	3
unixcoder	0.3492- >0.3598- >0.3685	0.5949- >0.6303- >0.632	0.3749- >0.4271- >0.4271	0.5593- >0.6002- >0.5954
codet5	0.3225- >0.4004- >0.4096	0.6181- >0.6147- >0.6147	0.4325- >0.4313- >0.434	0.5416- >0.5427->0.55
cct5	0.3176- >0.3158- >0.3122	0.6787- >0.6787- >0.6787	0.4185- >0.4605- >0.4605	0.5546- >0.5686- >0.5666

Project	1	2	3	4
codebert	0.337->0.4192- >0.4162	0.5361- >0.5231- >0.5273	0.4831- >0.4698- >0.4824	0.4688- >0.4983- >0.5037
plbart	0.3967- >0.4214- >0.4242	0.588->0.5883- >0.5913	0.5064- >0.5065- >0.5219	0.4847- >0.5654- >0.5503
graphcodebert	0.364->0.3975- >0.3936	0.5539- >0.5539- >0.5539	0.5203- >0.5272- >0.535	0.507->0.5434- >0.5386
unixcoder	0.3982- >0.4165- >0.3984	0.5569- >0.5717- >0.5717	0.4757->0.538- >0.5349	0.4601- >0.4601- >0.4601
codet5	0.3644- >0.4052- >0.4074	0.5631- >0.5648- >0.5625	0.5224- >0.5528- >0.5554	0.5488- >0.4732- >0.495
cct5	0.3813->0.396- >0.3948	0.4539- >0.4717- >0.4712	0.5038- >0.5907- >0.5907	0.4999- >0.5483- >0.5357

Developer	1	3	
codebert	0.5031->0.5904->0.5782	0.2777->0.3605->0.3379	
plbart	0.467->0.5279->0.5157	0.4143->0.4336->0.4378	
graphcodebert	0.5095->0.5952->0.554	0.3515->0.392->0.3807	
unixcoder	0.3789->0.49->0.4451	0.3398->0.3907->0.3792	
codet5	0.4747->0.5133->0.5036	0.3625->0.3738->0.3663	
cct5	0.5531->0.5959->0.5959	0.3577->0.3852->0.387	

结果显示，当某些簇的性能在Lora上下降时，加入OutlierDetector可以缓和其下降趋势，甚至重新恢复性能提升。

RQ3: 工作量感知指标，Effort--Aware评估

工作量感知指标表示Kmean方法时有效的，但是同时另外两组方法的性能却反而下降了，这是为何？

R20E	CodeBert	Plart	GraphCodeBert	UnixCoder	CodeT5	CCT5
base	0.7743	0.782	0.7785	0.7791	0.7579	0.8076
developer_aware	0.7747	0.784	0.7794	0.7811	0.7524	0.8084
Kmean	0.7806	0.7899	0.7878	0.7878	0.784	0.8131
project_final	0.7768	0.7869	0.7747	0.7874	0.7642	0.8122
JITLine	0.7053	**	**	**	**	**
CC2VEC	0.6295	**	**	**	**	**

E20R	CodeBert	Plart	GraphCodeBert	UnixCoder	CodeT5	CCT5
base	0.0129	0.0118	0.0128	0.012	0.0127	0.0104
developer_aware	0.0133	0.012	0.0132	0.0122	0.0141	0.0105
Kmean	0.0116	0.0114	0.0116	0.0115	0.0116	0.0103
project_final	0.013	0.0117	0.013	0.0119	0.0132	0.0096
JITLine	0.104	**	**	**	**	**
CC2VEC	0.197	**	**	**	**	**

Popt	CodeBert	Plart	GraphCodeBert	UnixCoder	CodeT5	CCT5
base	0.9194	0.9238	0.9203	0.922	0.9068	0.9337
developer_aware	0.9209	0.9257	0.92	0.922	0.9027	0.9344
Kmean	0.9217	0.9261	0.9252	0.9265	0.92	0.9355
project_final	0.9184	0.925	0.9175	0.9261	0.9083	0.9358
JITLine	0.8826	**	**	**	**	**
CC2VEC	0.824	**	**	**	**	**

分析可发现，此处工作量感知用的是缺陷密度，所以和1类的召回率非常强相关，下面给出召回率如下

Recall	codebert	plbart	graphcodebert	unixcoder	codet5	cct5
base	0.4661	0.4758	0.5246	0.4978	0.5196	0.5109
developer_aware	0.4813	0.4973	0.5398	0.5234	0.5149	0.5356
Kmean	0.5895	0.6017	0.6067	0.5865	0.6143	0.6135
project_final	0.5941	0.6139	0.6232	0.5406	0.6219	0.6135

可以发现EffortAware的性能差异主要和Recall相关。总而言之，在EffortAware的考虑下更建议使用Kmean方法。

讨论

目前，这份工作的贡献可以概括为：

1. 证明了专家特征可以作为代码变更风格聚类的特征，可以有效地将风格类似的代码划分到一起。
2. 提出了一种StyleAware的框架，在此框架上对6组预训练模型进行了实验，并被证明是有效的。
3. 研究探索了三种策略对代码风格聚类的不同表现，以及其适用场景（Developer适合在开发者数据足够时进行划分，对单个簇表现较好；Kmean适合代码项目和开发者不明的数据集，仅靠专家特征分布进行划分，在Effort Aware上表现较好；Project适合存在多种项目的数据集）。

1. Chao Ni, Wei Wang, Kaiwen Yang, Xin Xia, Kui Liu, and David Lo. 2022. The best of both worlds: integrating semantic features with expert features for defect prediction and localization. In Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE 2022). Association for Computing Machinery, New York, NY, USA, 672–683. <https://doi.org/10.1145/3540250.3549165>. [↗](#) [↗](#)

2. [1] Liu S , Keung J , Yang Z ,et al.Delving into Parameter-Efficient Fine-Tuning in Code Change Learning: An Empirical Study[J].IEEE, 2024.DOI:10.1109/SANER60148.2024.00055. [↗](#) [↗](#)

3. Pornprasit C, Tantithamthavorn C K. Jitline: A simpler, better, faster, finer-grained just-in-time defect prediction[C]//2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR). IEEE, 2021: 369-379. [↗](#)

4. Hoang T, Kang H J, Lo D, et al. Cc2vec: Distributed representations of code changes[C]//Proceedings of the ACM/IEEE 42nd international conference on software engineering. 2020: 518-529. [↗](#)

5. in B, Wang S, Liu Z, et al. Cct5: A code-change-oriented pre-trained model[C]//Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2023: 1509-1521. [↗](#)